

Last modified on

Sports Data Soccer API - Fixtures And Results Feed (MA1_Detailed)

Get a fixture or fixture list with match details, such as date, start time, contestants, competition, season, score, result, lineups, and Power Rating information for both the contestants (teams) involved in a match.

Overview

The Soccer **Fixtures and Results Detailed feed** provides a detailed fixture list and can contain the following match information:

- Date and kick-off time
- Home and Away contestants (with IDs)
- Competition and Season
- Venue
- Score and Match Result
- Key Events (Goals, Assists, Penalties, Cards, Substitutions)
- Following Power Rating information for contestants: `currentPowerRating` , `highestSeasonRating` , `lowestSeasonRating` , `averageSeasonRating` , `currentCompetitionRank` , `currentConfederationRank` and `currentGlobalRank`

Update - Behind Closed Doors and Limited Audience matches:

We have implemented a way to highlight when a match will be played behind closed doors or with a limited audience. When these conditions apply, new attributes for `attendanceInfoId` and `attendanceInfo` will be delivered in the feed under the `matchInfo` element. For more information, see: **Feed output**

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 11+13	TIER 8+9+10	TIER 6+7	TIER 5	TIER 1+2+3
Updated on any update to the match time, date, teams, competition, round, season, or key event (lineup, card, goal, substitution, assist)	See Tier 13	Post-match	No more frequently than one URL poll every 60 seconds for range: <KO - 1h; KO + 4h> Up to 1-hour interval for range: <KO + 4h; KO + 8h> Up to 24-hour interval 8h+ of match ending, lasting a maximum of 7 days					One poll ~12-24 hours from the end of each game (it can vary)

Note: Please consider using the **Delta parameter** feature before integration as this can hugely improve feed performance

Use Deltas To Get Updates

Get updated data and make streamlined requests using the Delta parameter

We recommend that you use the Delta (`_dltr`) parameter in your requests, so that you can programmatically get the latest assets since a specified date/time. It could significantly benefit and streamline your use of the SDAPI, reduce your polling overheads, and improve latency and overall performance. This allows you to return and ingest only the data that has changed in a specific time period from the feed (instead of receiving all data and checking each individual match). You can use the Delta parameter with existing queries, such as filtering by competition, tournament calendar, and match date/time. See [Query parameters and example request parameters](#) for an example.

Fixtures And Results (MA1_detailed) Guide

USAGE	
GET detailed information for a list of Fixtures and Results	/socce
GET information for one or more individual Fixtures and Results by UUID (<code>fx</code> parameter - see the Query parameters below)	/socce &{quer
GET detailed information for a specific fixture	/socce {query
GET detailed Fixtures and Results data according to specified parameter values	/socce
GET detailed information for a list of Fixtures and Results associated with a tournament calendar UUID	/socce &{quer

Note: You can call the MA1 feed without a Tournament Calendar or Match UUID - this returns the entire list (first page) to which you are subscribed, sorted by kick-off time in descending order (default for all queries).

How can I get fixture and other entity UUIDs?
Use the [Tournament Calendar \(OT2\)](#) or [Fixtures and Results \(MA1_detailed\)](#) feeds to get fixture metadata, including UUIDs for each fixture and its linked contestants, competition, tournament calendar and other entities.

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you can query the `/matchdetailed` feed without an entity UUID.

Legacy IDs: You can use the following query parameters to filter by legacy IDs.

- `fx` - only for a single match; the comma-separated list supports fixture UUIDs but does not support legacy IDs
- `comp` + `mt.mDt` (when used in combination with each other, not separately)
- `ctst` + `mt.mDt` (when used in combination with each other, not separately)

Important: If you call the feed using multiple filters by ID then only one type of ID can be passed to all parameters - this means choosing to use either only the 'UUID' type, or all 'oc id' type, or all 'op id' type (for example, all 'UUID' type: `comp=uuid&stg=uuid`). You cannot mix ID types (for example, `comp=uuid&stg=urn:perform:optacore:stage:ocid`); otherwise, the feed will return an error.

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example GET requests (rather than cURL, for example). All URIs are case-sensitive - this means that URI strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` (Outlet Authorisation Key), query parameters for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format). If required, it should also include an entity UUID (such as a fixture/match UUID).
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authorisation Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{}` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>_dlr</code>	Programmatically get the latest fixture and result data assets since a particular date/time (created, updated and published), and subject to your specified query parameters. Allows users to only query matches where data has changed. What is the benefit? It streamlines the queries while improving performance and latency, because you are able to know in which specific matches there have been updates - this is instead of polling a feed for a whole competition (or multiple competitions) and having to check each individual match to find out. You can also use all other filters, such as: <code>tmc1</code>, <code>comp</code>, <code>mt.mDt</code> There is no asset versioning system, but you can use the Delta parameter to ingest updates. Value:

Query parameter	Value and description
	Use {xsd:dateTime} (with timezone) or {milliseconds} (since the epoch) to specify the date and time. The format of the date/time for XSD UTC/GMT is: YYYY-MM-DDThh:mm:ss%2B00:00 <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&_dlt={dateTime}&{queryParameters}</pre>
fx	GET detailed information about one fixture or multiple fixtures. - To return one fixture: Pass a fixture UUID or legacy Opta match ID to the fx parameter. - To return multiple fixtures: Pass the fixture UUIDs in a comma-separated list to the fx parameter. For the B2B outlet request type (_rt=b) you can pass up to 50 UUIDs, while a B2C request (_rt=c) allows up to 20 UUIDs. Multiple legacy Opta and Opta Core match IDs are not supported. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx={fixtureUuid}</pre>
fixture/match UUID	GET detailed information about one fixture by passing the fixture/match UUID as a path parameter (the asset/resource) in the URL. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}</pre>
_fld	GET only the subsets of data fields that you specify, instead of all points, by passing specific values to the _fld parameter in the URL. This also reduces the size of the feed, loading speed, and only pulls back the relevant, specified data. All existing MA1 parameters can be used in combination with _fld. Values of _fld parameter are based on position of fields in the XML. They are shortcuts of the relevant path to the field; therefore, once you take a look, you should be able to use the following logic to get the values: - first two letters of every word of a field name - eg main.da = matchInfo + date - first three letters of a word for one-word parts of path - eg main.con.con.id = matchInfo + contestant + contestant + id - first two letters of each word for multi-word parts of the path - eg main.da = matchInfo + date Example: The call returns matchInfo element (main), match date and ID (main.da), each contestant ID and position (main.con.con.id), competition, and country ID (main.com.cou.id): <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&_fld=main.da,main.con.con.id,main.com.cou.id</pre>
comp	GET information about fixtures and results only for a specific competition. Pass the competition UUID to the comp parameter - you can pass one or up to 50 competition UUIDs in a comma-separated list. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&comp={competitionUuid},{competitionUuid}</pre>

Query parameter	Value and description
comp + ctst	GET information about fixtures and results only for a specific contestant in a competition. Pass the contestant UUID to the ctst parameter, and competition UUID to the comp parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&comp={competitionUuid}&ctst={contestantUuid}</pre>
comp + mt.mDt	GET fixtures and results only for the specified competition across all competitions in a specified date range - the maximum span of the date range is one year. Pass the date range (in the format YYYY-MM-DDTHH:MM:SSZ including [] square brackets) to the mt.mDt parameter and competition UUID to the comp parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&comp={competitionUuid}&mt.mDt=[YYYY-MM-DDTHH:MM:SSZ TO YYYY-MM-DDTHH:MM:SSZ]</pre>
ctst + ctstpos	GET fixtures and results filtered by whether the specified contestant will play in the home or away position, across all competitions. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&ctstpos{home away}</pre>
ctst (1) + ctst2 (2) + ctstpos	GET fixtures and results between the two specified contestants, across all competitions. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&ctst2={contestantUuid}</pre>
ctst (1) + ctst2 (2) + ctstpos	GET fixtures and results between the two specified contestants where the first contestant (1) plays in the home or away position, across all competitions. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&ctst2={contestantUuid}&ctstpos{home away}</pre>
ctst (1) + ctst2 (2) + mt.mDt	GET fixtures and results between the two specified contestants, across all competitions in a specified date range - the maximum span of the date range is one year. Pass the date range (in the format YYYY-MM-DDTHH:MM:SSZ including [] square brackets) to the mt.mDt parameter and contestant UUID to the ctst parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&ctst2={contestantUuid}&mt.mDt=[YYYY-MM-DDTHH:MM:SSZ TO YYYY-MM-DDTHH:MM:SSZ]</pre>
ctst + mt.mDt	GET fixtures and results only for the specified contestant across all competitions in a specified date range - the maximum span of the date range is one year. Pass the date range (in the format YYYY-MM-DDTHH:MM:SSZ including [] square brackets) to the mt.mDt parameter and contestant UUID to the ctst parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&mt.mDt=[YYYY-MM-DDTHH:MM:SSZ TO YYYY-MM-DDTHH:MM:SSZ]</pre>
lineups	GET only lineups information (including substitutes) for contestants in fixtures - you must use this parameter in combination with the live parameter with a value of yes. Data is returned in the <code><liveData/></code> element. The available values for the live and lineups parameters are: no

Query parameter	Value and description
	(default) and <code>yes</code> . We recommend filtering the live data by an entity UUID and parameter, such as a tournament calendar (<code>tmcl</code>) or contestant (<code>ctst</code>). <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&live={yes no}&live=yes&tmcl=bnni2f2e6ii20b1wmzcsbpj2o</pre>
<code>live</code>	GET live data (such as score, goals, cards, attendance, officials etc) for fixtures. If there is no live data available, then an empty <code><liveData/></code> element is returned. The available values for the <code>live</code> parameter are: <code>no</code> (default) and <code>yes</code> . If the parameter is not used, the default behaviour of the feed is not to return live data (equivalent to <code>live=no</code>). However, once the match is over, the default response will have live data, irrespective of whether the feed is queried with <code>live=yes</code> or not. Note: We recommend filtering the live data by an entity UUID and parameter, such as a tournament calendar (<code>tmcl</code>) or contestant (<code>ctst</code>). Example: <code>tmcl</code> UUID + <code>live</code> <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&live={yes no}&tmcl=bnni2f2e6ii20b1wmzcsbpj2o</pre>
<code>mt.mDt</code>	GET fixtures and results only for the specified match date range - the maximum span of the date range is one year. Pass the date range (in the format <code>YYYY-MM-DDTHH:MM:SSZ</code>) to the <code>mt.mDt</code> parameter. You can use this parameter in combination with other filter parameters (including <code>comp</code> , <code>ctst</code> , <code>srs</code> , <code>stg</code> , <code>tmcl</code>). (Note: Remove the placeholder value curly braces <code>{}</code> from your call) <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&ctst={contestantUuid}&mt.mDt=[YYYY-MM-DDTHH:MM:SSZ TO YYYY-MM-DDTHH:MM:SSZ]</pre>
<code>prsn + prsntp</code>	GET only matches in which a given person appears either as a player or a team official - this depends on the <code>prsntp</code> parameter. The value of the <code>prsn</code> parameter is a UUID of a person. Note: You must use this parameter in combination when all of the following parameter and value pairs are present: <code>live=yes</code> & <code>lineups=yes</code> & <code>prsntp={player teamofficial}</code> <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&live=yes&lineups=yes&prsntp={personType}&prsn={personUuid}</pre>
<code>srs</code>	GET information about fixtures and results only for a specific series. Pass the series UUID to the <code>srs</code> parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&srs={seriesUuid}</pre>
<code>status (+ live)</code>	GET only games of a certain match status. Data is returned in the <code><liveData/></code> element. Note: You must use this parameter in combination with the following parameters; otherwise the feed will return an error: <code>live</code> (a parameter-value of <code>live=yes</code>) - an entity parameter and UUID such as <code>tmcl={tournamentCalendarUuid}</code> (a tournament calendar) The default behaviour is all types (equivalent to <code>status=all</code>). You can pass only one value to the <code>status</code> parameter. Available values: <code>Fixture</code> , <code>Played</code> , <code>Playing</code> , <code>Cancelled</code> , <code>Postponed</code> , <code>Suspended</code> , <code>All</code> (default) Note: In the Soccer API, the values for <code>status</code> are not case-sensitive - for example, both <code>status=fixture</code> and <code>status=Fixture</code> are valid.

Query parameter	Value and description
	<pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&status={statusType}&live=yes&tmcl=bnni2f2e6ii20b1wmzcsbpj2o</pre>
stg	GET information about fixtures and results only for a specific stage of a tournament calendar. Pass the stage UUID to the <code>stg</code> parameter, <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&stg={stageUuid}</pre>
tmcl	GET information about fixtures and results only for the specified tournament calendar. Pass the tournament calendar UUID to the <code>tmcl</code> parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&tmcl={tournamentCalendarUuid}</pre>
tmcl + ctst	GET information about fixtures and results only for a specific contestant of a specified tournament calendar. Pass the contestant UUID to the <code>ctst</code> parameter, and tournament calendar UUID to the <code>tmcl</code> parameter. <pre>GET http://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&tmcl={tournamentCalendarUuid}&ctst={contestantUuid}</pre>
tmcl + stg	GET information about fixtures and results only for a specific stage of a tournament calendar. Note: You can use the <code>stg</code> parameter without <code>tmcl</code>. Pass the stage UUID to the <code>stg</code> parameter, and tournament calendar UUID to the <code>tmcl</code> parameter, if you require it. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&tmcl={tournamentCalendarUuid}&stg={stageUuid}</pre>
week	GET information about fixtures and results only for the specified game week. Pass the week value (<i>integer</i>) to the <code>week</code> parameter, for example <code>1</code>, <code>2</code>, and so on. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&week={weekNumber}</pre>
cvlv	GET fixtures for only the specified coverage level (subject to other query parameters/criteria). This will filter out (hide) all matches that are not from requested coverage level. Pass one value (or multiple values in a comma-separated list) to the parameter: <code>1</code>, <code>2</code>, <code>3</code>, <code>4</code>, <code>5</code>, <code>6</code>, <code>7</code>, <code>8</code>, <code>9</code>, <code>10</code>, <code>11</code>, <code>12</code>, <code>13</code>, <code>14</code>, <code>15</code> - this is a number that represents the coverage level tier, in the range <code>1 - 15</code> - for example: <code>cvlv=9</code> Note: - This parameter is not supported when the call contains a single fixture UUID (to the <code>fx</code> parameter or as the path parameter). It is supported with all other parameters. - If no matches are available for that tier and query criteria, the standard error code is returned to reflect that no data is available. - If this parameter is not used the default behaviour of the feed is to return fixtures from all tiers, subject to other query criteria. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&cvlv={coverageLevel}</pre>

Query parameter	Value and description
<code>_ordSrt</code>	GET fixtures and results with sorted by match date/time in ascending or descending order. <code>_ordSrt=asc</code> - the resulting list is sorted by match date/time ascending. <code>_ordSrt=desc</code> - the resulting list is sorted by match date/time descending. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&_ordSrt={asc desc}</pre>
<code>_pgNm</code> + <code>_pgSz</code>	GET data only in the specified page size and/or page number. All results are returned in pages and these parameters help limit the page size in order to optimise the efficiency of the SDAPI. <code>_pgNm</code> - Specify the <code>{pageNumber}</code> to return - indexed from <code>1</code>. If this value is not provided, then page 1 is returned by default. <code>_pgSz</code> - Specify the size of the page to return - the number of match definitions in a page. Valid <code>{pageSize}</code> values depend on the operating mode of an outlet. For B2B requests, the maximum value is <code>1000</code> (default is 20). For B2C requests, the maximum value is <code>50</code> (default is 20). <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_pgSz={pageSize}&_pgNm={pageNumber}&tmcl=bnni2f2e6ii20b1wmzcsbpj2o</pre>
<code>_lcl=en-op</code>	GET assets with the Opta namings in English in the response (instead of Opta Core namings). When the call does not include the <code>_lcl=en-op</code> parameter-value, the feeds will continue to work as they do currently; one example is that the response will be in the default language of English unless the call contains the <code>_lcl</code> parameter and a locale code value to request the response in a different locale/language. Note: You can use this parameter in combination with other valid parameters and values in the request. However, you cannot use it in combination with a locale/language code and <code>_lcl</code> parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchdetailed/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&_lcl=en-op&{queryParameters}</pre>

Do not supply both the `_lcl` (with a locale/language code value) and `_lcl=en-op` in the same call - this is not supported; otherwise, the feed will return an error and code `10203` (Ambiguous/Invalid request parameters) in the response due to a duplicated `_lcl` parameter.

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML - single game, basic](#)
- [XML - single game \(fx\), basic](#)
- [XML - season \(tmcl\), basic](#)
- [JSON - season \(tmcl\), basic](#)
- [XML - season \(tmcl\), detailed](#)
- [JSON - season \(tmcl\), detailed](#)
- [XML - contestant1 \(ctst\) vs contestant2 \(ctst2\)](#)
- [XML - contestant1 \(ctst\) vs contestant2 \(ctst2\), contestant \(1\) in home position \(ctstpos="home"\)](#)

Example (`_dlt`):


```
http://api.performfeeds.com/soccerdata/matchDetailed/1vmmaetzoxxkgg1qf6pkpfmku0k/bsu6pjne1eqz2hs8r3685vbhl?_fmt=xml
&_rt=b&_dlt=2012-06-07T07:00:00%2000:00
```

The `_dlt` parameter is intended to be used in applications that periodically poll the feed for updates, and should be given a value that corresponds to the date/time of the previous poll to the same feed. By integrating the `_dlt` parameter in your code, you can automate polling, retrieving and publishing updated assets from the feed.

Important: Make sure that there is no gap in the frequency between requests for new assets and the Delta time period value used to request updated assets from; otherwise, you will not capture any updates/changes made to assets, including those you have previously retrieved.

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

Note:

- Each response provides match details for a fixture. However, you will only receive additional lineup details, scores, and other 'live' data on request by passing these parameter-values in the call: `live=yes` (live data) or `live=yes&lineups=yes` (live and lineups data).
- The root element can be `matches` or `match` - see the table below.
- In the scenario where the match has not yet been played (for example, it is a scheduled fixture to be played in the future), not all attributes listed here will be present and we may provide placeholder values for some attributes. For example, the scores, winner, and contestant names, position, and ID may not be known (the contestant `id` attribute may also not be returned). However, the feed will be updated with this information once it is available.

▶<matchDetailed> (XML only)

▶<match>

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<liveData>

▶<matchDetails>

▶<periods> (XML only)

▶<period>

▶<suspensions>

▶<suspension>

▶<scores>

▶<ht>

▶<ft>

▶<et>

▶<pen>

▶<total>

▶<aggregate>

▶<totalUnconfirmed>

▶<goal>

▶<missedPen>

▶<card>

▶<substitute>

▶<VAR>

▶<penaltyShootout>

▶<penaltyShot>

▶<lineUp>

▶<player>

▶<teamOfficial>

▶<matchDetailsExtra>

▶<attendance>

▶<awayAttendance>

▶<matchOfficials> (XML only)

▶<matchOfficial>

How Are VAR Events Reported?

An update to the feed means that when a VAR (Video Assistant Referee) review takes place our analysts will collect the type of event being reviewed, the team and player involved, and the decision of the review (outcome) which also describes what happens next.

Outcomes

The following table gives a breakdown of possible outcomes from a VAR review, according to the VAR event type/reason and the referee decision.

The outcomes appear in the **VAR** element of the feed response. For a list of all VAR-related attributes that may appear during/after a match, see the **VAR elements/attributes** and **Feed output** sections in this guide.

- **type** (VAR element) - describes the original event/decision that is the reason for the VAR review
- **outcome** (VAR element) - the outcome of the VAR review; the value is the event that stands in the match
- **decision** (VAR element) - the referee decision following the VAR review, in relation to the original decision: **Cancelled** (original event cancelled, the 'outcome' value gives the event that will stand); **Confirmed** (original event confirmed and stands, as per the 'outcome' value)

VAR event (varReason value, matchDetails node /type value, VAR node)	Referee decision (decision attribute value)	Outcome (outcome attribute value)	Explanation
Goal awarded *	Confirmed	Goal	<i>Original decision is not changed after review</i>
Goal awarded *	Cancelled	No goal	<i>Original decision is changed after review</i>
Goal not awarded *	Confirmed	No goal	<i>Original decision is not changed after review</i>
Goal not awarded *	Cancelled	Goal	<i>Original decision is changed after review</i>
Penalty awarded	Confirmed	Penalty	<i>Original decision is not changed after review</i>
Penalty awarded	Cancelled	No penalty	<i>Original decision is changed after review</i>

VAR event (varReason value, matchDetails node /type value, VAR node)	Referee decision (decision attribute value)	Outcome (outcome attribute value)	Explanation
Penalty not awarded	Confirmed	No penalty	Original decision is not changed after review
Penalty not awarded	Cancelled	Penalty	Original decision is changed after review
Red card given	Confirmed	Red card	Original decision is not changed after review
Red card given	Cancelled	No red card	Original decision is changed after review
Card upgrade	Confirmed	No change	Original decision is not changed after review
Card upgrade	Cancelled	Card upgraded	Original decision is changed after review
Mistaken identity	Confirmed	Player originally cautioned/ sent off confirmed	Original decision is not changed after review
Mistaken identity	Cancelled	Ref cautions/ sends off a different player	Original decision is changed after review
Other	Cancelled	n/a	Original decision is not changed after review
Other	Confirmed	n/a	Original decision is changed after review

VAR Elements/Attributes

The following table describes the different ways that we use VAR attributes. For more information, see the relevant elements in the **Feed output** section.

DESCRIPTION
VAR review flag - These attributes show that: 1) there is a delay 2) the reason (incident type) requiring the review 3) the relevant to - They are only shown in feed when a VAR review is taking place, the attributes and values disappear from the feed
VAR information for goal events - These attributes show that: 1) a goal has been awarded following a review, and; 2) whether the goal was: a) originally awarded and confirmed after review, or b) only awarded following a review

DESCRIPTION

- **Note:** The player attached to a VAR Goal event may not always be the scorer of the goal under review. In some c

All VAR events in a game

- This element will appear in any game where there has been a VAR review.
- It will list all VAR events in a game, the player and team involved, the type of event reviewed, whether the original
- This list will update during a game as more events are reviewed.

